

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

on

Artificial Intelligence (23CS5PCAIN)

Submitted by

Pranav Hegde (1BM22CS202)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Artificial Intelligence (23CS5PCAIN)” carried out by **Pranav Hegde (1BM22CS202)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Artificial Intelligence (23CS5PCAIN) work prescribed for the said degree.

Prof. Sneha S Bagalkot

Assistant Professor

Department of CSE, BMSCE

Dr. Kavitha Sooda

Professor & HOD

Department of CSE, BMSCE

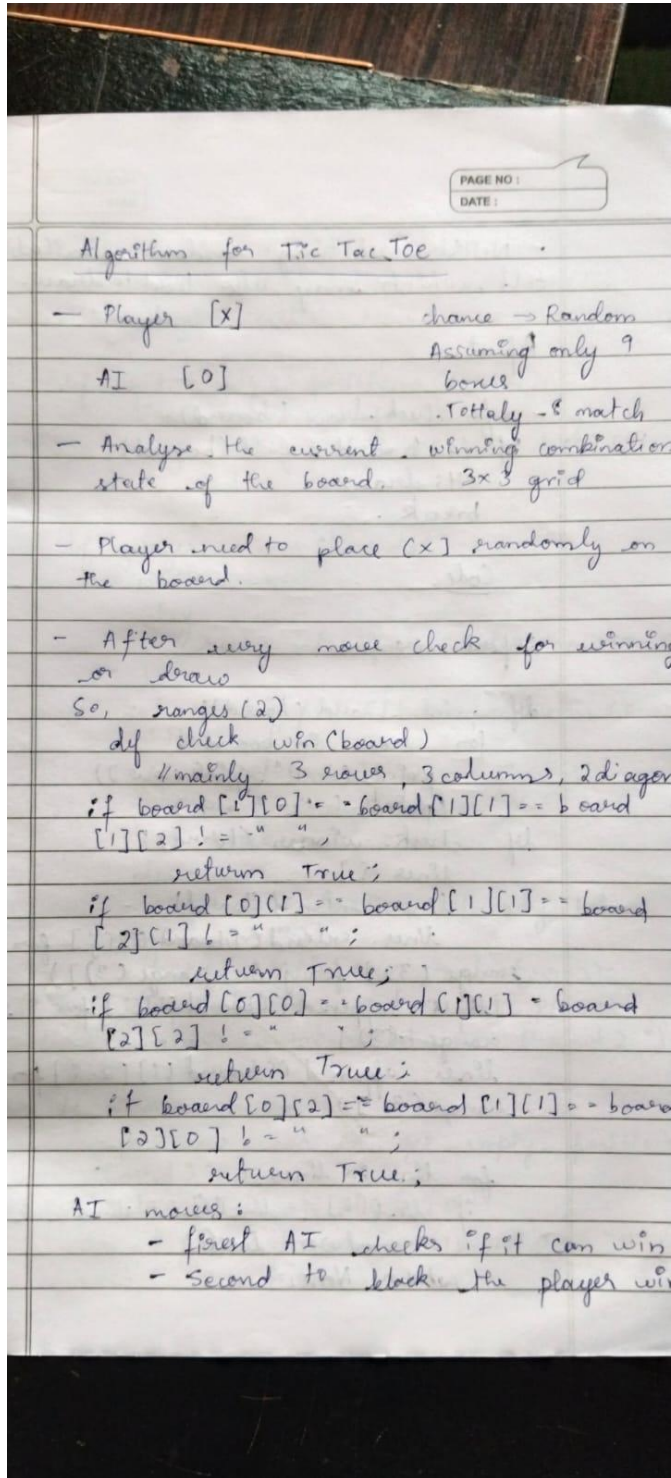
Index

Sl. No.	Date	Experiment Title	Page No.
1	4-10-2024	Implement Tic –Tac –Toe Game Implement vacuum cleaner agent	1-7
2	18-10-2024	Implement 8 puzzle problems using Depth First Search (DFS) Implement Iterative deepening search algorithm	8-15
3	25-10-2024	Implement A* search algorithm	16-22
4	8-11-2024	Implement Hill Climbing search algorithm to solve N-Queens problem	23-25
5	15-11-2024	Simulated Annealing to Solve 8-Queens problem A* to Solve 8-Queens problem	26-28
6	22-11-2024	Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.	29-30
7	29-12-2024	Implement unification in first order logic	31-34
8	6-12-2024	Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.	35-37
9	6-12-2024	Create a knowledge base consisting of first order logic statements and prove the given query using Resolution	38-40
10	13-12-2024	Implement MinMax Algorithm for TicTacToe Implement Alpha-Beta Pruning.	41-43

Github Link: <https://github.com/pranav0hegde/AI>

Implement Tic-Tac-Toe Game.

Algorithm:



Code:

```
import random
import numpy as np

board = ["-" * 3 for _ in range(3)]

def check_win():
    for i in range(3):
        if board[i][0] == board[i][1] == board[i][2] != "-":
            return True
        if board[0][i] == board[1][i] == board[2][i] != "-":
            return True

    if board[0][0] == board[1][1] == board[2][2] != "-":
        return True
    if board[0][2] == board[1][1] == board[2][0] != "-":
        return True

    return False

def full():
    return all(cell != "-" for row in board for cell in row)

def can_win(m):
    for i in range(3):
        row = board[i]
        if row.count(m) == 2 and row.count("-") == 1:
            return (i, row.index("-"))
```

```

    for i in range(3):
        col = [board[j][i] for j in range(3)]    if
col.count(m) == 2 and col.count("-") == 1:
        return (col.index("-"), i)

    diag1 = [board[i][i] for i in range(3)]    if
diag1.count(m) == 2 and diag1.count("-") == 1:
        return (diag1.index("-"), diag1.index("-"))

    diag2 = [board[i][2 - i] for i in range(3)]    if
diag2.count(m) == 2 and diag2.count("-") == 1:
return (diag2.index("-"), 2 - diag2.index("-"))

return None

def display():
    print(np.array(board))

while True:
    display()
    u = tuple(map(int, input("Enter row and column for X (0-2): ").strip().split()))
    if board[u[0]][u[1]] != "-":
        print("Invalid move, try again.")
    continue

    board[u[0]][u[1]] = "X"

```

```

        if check_win():
display()      print("X
wins!")      break
        if full():
display()
print("It's a tie!")
break

        move = can_win("O")
print(move)    if move
is None:
        move = can_win("X")
if move is None:
        empty = [(i, j) for i in range(3) for j in range(3) if board[i][j] == "-"]
move = random.choice(empty)    if board[1][1]=="-":
        move=(1,1)
board[move[0]][move[1]] = "O"

        if check_win():
display()      print("O
wins!")      break

```

Output:

```
[[ '-' '-' '-']
[- '-' '-']
[- '-' '-']]
Enter row and column for X (0-2): 0 0
None
[['X' '-' '-']
[- '0' '-']
[- '-' '-']]
Enter row and column for X (0-2): 0 1
None
[['X' 'X' '0']
[- '0' '-']
[- '-' '-']]
Enter row and column for X (0-2): 1 0
(2, 0)
[['X' 'X' '0']
['X' '0' '-']
['0' '-' '-']]
0 wins!
```

Implement Vacuum Cleaner Agent.

Algorithm:

DATE: 1/10/24

Vacuum Cleaner (2 Room)

```
for (room = 0)
    vacuum
    if
        (check, left)
    else (check, Right)
```

The diagram illustrates the movement of a vacuum cleaner agent between two rooms. It consists of four 2x2 grids representing rooms, connected by arrows. The top-left grid shows the agent (a circle with a dot) in the bottom-left cell, with a small circle in the top-left cell. An arrow points to the top-right grid, where the agent is in the bottom-left cell and the small circle is in the top-right cell. A downward arrow points to the bottom-right grid, where the agent is in the bottom-right cell and the small circle is in the top-right cell. A leftward arrow points to the bottom-left grid, where the agent is in the top-right cell and the small circle is in the top-right cell.

Taking Inputs

- Rooms
- Dirty / Clean
- Left, Right.

```
for (room = Dirty)
    vacuum
    if (check, left)
```

Code: import

random

```
l=[random.choice([0,1]),random.choice([0,1])]
```

```
def check(i):    if l[i]==0:        l[i]=1
```

```
    print(f'Cleaned Room {i}')
```

```
print(f'Moved to Room {(i+1)%2}')
```

```
return (i+1)%2 i=random.choice([0,1])
```

```
print(f'{i} is the start index') print("0
```

```
is dirty and 1 is clean") print(f'{1} is
```

```
the initial state of room")
```

```
while sum(l)!=2:
```

```
i=check(i)    if
```

```
l[(i+1)%2]==1:
```

```
    l[(i+1)%2]=random.choice([0,1])
```

```
if l[(i+1)%2]==0:
```

```
    print(f'Room {(i+1)%2} got dirty')
```

```
print(f'{1} is current state of rooms') print("Rooms
```

```
are clean")
```

Output:

```
1 is the start index
0 is dirty and 1 is clean
[0, 0] is the initial state of room
Cleaned Room 1
Moved to Room 0
Room 1 got dirty
[0, 0] is current state of rooms
Cleaned Room 0
Moved to Room 1
Room 0 got dirty
[0, 0] is current state of rooms
Cleaned Room 1
Moved to Room 0
Room 1 got dirty
[0, 0] is current state of rooms
Cleaned Room 0
Moved to Room 1
Room 0 got dirty
[0, 0] is current state of rooms
Cleaned Room 1
Moved to Room 0
[0, 1] is current state of rooms
Cleaned Room 0
Moved to Room 1
Room 0 got dirty
[0, 1] is current state of rooms
Moved to Room 0
Room 1 got dirty
[0, 0] is current state of rooms
Cleaned Room 0
Moved to Room 1
[1, 0] is current state of rooms
Cleaned Room 1
Moved to Room 0
[1, 1] is current state of rooms
Rooms are clean
```

Implement 8 puzzle problems using Depth First Search (DFS).

Algorithm:

DATE: 15/10/24

Q Solving 8 Puzzle problem using A*
Q Implementing Iterative Deeping Search Algorithm.

Step 1: • Initialize the problem
• Define the initial state in a puzzle of (3x3).
• Define goal state in puzzle of 3x3
determining empty tile $\rightarrow 0$

Q1

initial state			Final state		
1	2	3	2	8	1
8	0	4	0	4	3
7	6	5	7	6	5

Let lowest Value using priority queue
Calculating each tile from initial to final

	V	H	Dist	Pri. Q
1	0	2	-2	2
2	0	1	1	1
3	1	0	1	1
4	1	1	2	1
8	0	1	1	2
0	0	0	0	
7	0	0	0	0
6	0	0	0	0
5	0	0	0	0

Applying this priority queue.

State moves

$[(0,1), (0,-1), (1,0), (-1,0)]$

Initial state

1	2	3
8	0	4
7	6	5

→

1	0	3
8	2	4
7	6	5

↓

8	1	3
0	2	4
7	6	5

←

0	1	3
8	2	4
7	6	5

↓

8	1	3
2	0	4
7	6	5

→

8	1	3
2	4	0
7	6	5

↓

8	0	1
2	4	3
7	6	5

←

8	1	0
2	4	3
7	6	5

↓

0	8	1
2	4	3
7	6	5

→

2	8	1
0	4	3
7	6	5

goal state.

Code:

```
import heapq
import numpy as np

goal = [[0,1,2], [3,4,5], [6,7,8]]
vis = set()
q = []
parent_map = {}
move_map = {}

def manhattan(curr):
    ans = 0
    pos = {goal[i][j]: (i, j) for i in range(3) for j in range(3)}
    for i in range(3):
        for j in range(3):
            x, y = pos[curr[i][j]]
            ans += abs(i - x) + abs(j - y)
    return ans

def moves(curr):
    x, y = [(i, j) for i in range(3) for j in range(3) if curr[i][j] == 0][0]
    poss = [[0, -1, 'left'], [-1, 0, 'up'], [1, 0, 'down'], [0, 1, 'right']]
    for dx, dy, direction in poss:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            curr1 = [row[:] for row in curr]
            curr1[x][y], curr1[nx][ny] = curr1[nx][ny], curr1[x][y]
            tuple_curr1 = tuple(map(tuple, curr1))
            if tuple_curr1 not in vis:
                heapq.heappush(q, (manhattan(curr1), curr1))
                vis.add(tuple_curr1)
            parent_map[tuple(map(tuple, curr1))] = curr
            move_map[tuple(map(tuple, curr1))] = direction
```

```

def dfs(curr):
    vis.add(tuple(map(tuple, curr)))
    if curr == goal:    return True
    moves(curr)    if q:
        curr = heapq.heappop(q)[1]
    if dfs(curr):    return True
    return False

def display_board(board):
    print("+---+---+---+")    for
    row in board:
        print("| " + " | ".join(str(x) if x != 0 else ' ' for x in row) + " |")    print("+---+---
+---+")

c=[[[] for i in range(3)] for
i in range(3):
    print(f"Enter elements of row {i+1}")    c[i]=list(map(int,input().split())) dfs(c)

result_path = []
directions = []
state = goal while
state:
    result_path.append(state)
    directions.append(move_map.get(tuple(map(tuple, state)), None))
    state = parent_map.get(tuple(map(tuple, state)))

for ind, (state, direction) in enumerate(reversed(list(zip(result_path, directions)))):

```

```
    print(f"Step {ind}:")
display_board(state) if
ind==0:
    print("Initial state")
if direction:
    print(f" Move empty space {direction}")
print()

print(f"Steps taken: {len(result_path) - 1}")
```

Output:

```
Enter elements of row 1
3 7 6
Enter elements of row 2
4 5 8
Enter elements of row 3
2 0 1
```

Step 0:

```
+---+---+---+
| 3 | 7 | 6 |
+---+---+---+
| 4 | 5 | 8 |
+---+---+---+
| 2 |   | 1 |
+---+---+---+
```

Initial state

Step 1:

```
+---+---+---+
| 3 | 7 | 6 |
+---+---+---+
| 4 |   | 8 |
+---+---+---+
| 2 | 5 | 1 |
+---+---+---+
```

Move empty space up

Step 57:

```
+---+---+---+
| 3 | 1 | 2 |
+---+---+---+
| 4 |   | 5 |
+---+---+---+
| 6 | 7 | 8 |
+---+---+---+
```

Move empty space down

Step 58:

```
+---+---+---+
| 3 | 1 | 2 |
+---+---+---+
|   | 4 | 5 |
+---+---+---+
| 6 | 7 | 8 |
+---+---+---+
```

Move empty space left

Step 59:

```
+---+---+---+
|   | 1 | 2 |
+---+---+---+
| 3 | 4 | 5 |
+---+---+---+
| 6 | 7 | 8 |
+---+---+---+
```

Move empty space up

Steps taken: 59

Implement Iterative deepening search algorithm.

Algorithm:

IDF

Step 1: Initialize the tree with node and the parent leaf node.
Mention the initial and the start node and also the final destination node.

find the destination node first
find() {

 using BFS method():
 level after level search
 for destination node.

 if present
 return level
 else

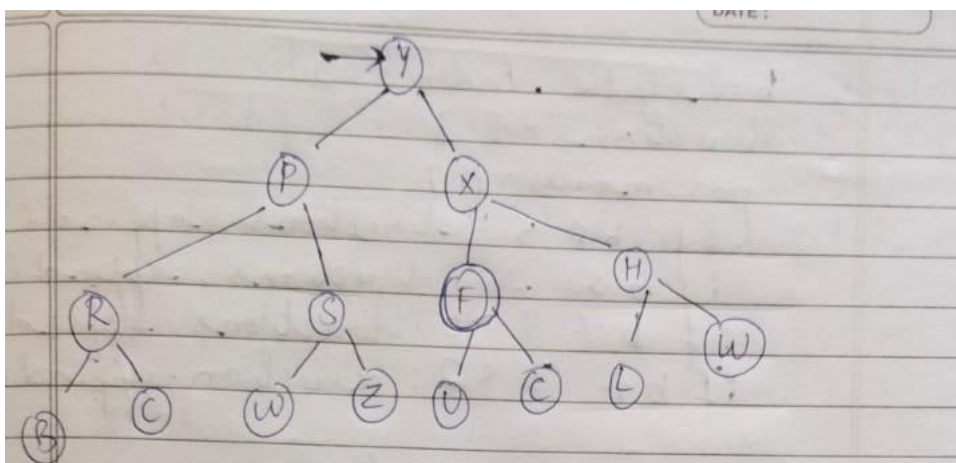
 go to next level.

find the parent node using back tracking

 find parent node()

 Backtrack the parent node
 to figure out the path.

Backtrack and print the node from
start to End.
if found.



Start node $\Rightarrow Y$ Destination $\Rightarrow F$

BFS :-

Step 1 $\rightarrow Y$

Step 2 $\rightarrow P, X$

Step 3 $\rightarrow R, S, \textcircled{F}$

Destination Node

Backtracking to $\textcircled{X} \rightarrow \textcircled{Y}$

Final State is

$\textcircled{Y} \rightarrow \textcircled{X} \rightarrow \textcircled{F}$

Code for A*

```

def H_n(state, target):
    return sum(x != y for x, y in zip(state, target))
def F_n(state_with, lvl, target):
    state, lvl = state_with, lvl
    return H_n(state, target) + lvl
  
```

Code:

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.children = [] # List to hold children nodes

    def add_child(self, child_node):
        self.children.append(child_node)

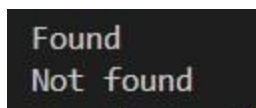
def iddfs(root, goal):
    for i in range(0,100000):
        res=dls(root,goal,i)
        if res:
            print("Found")
    return
    print("Not found")

def dls(root,goal,depth):
    if depth==0:
        if root.value==goal:
            return True
        return False
    for child in root.children:
        if dls(child,goal,depth-1):
            return True
    return False

root=TreeNode("Y") node1=TreeNode("P")
node2=TreeNode("X")
node3=TreeNode("R")
node4=TreeNode("S")
node5=TreeNode("F")
node6=TreeNode("H")
node7=TreeNode("B")
node8=TreeNode("C")
```

```
node9=TreeNode("S")
root.add_child(node1)
root.add_child(node2)
node1.add_child(node3)
node1.add_child(node4)
node2.add_child(node5)
node2.add_child(node6)
node3.add_child(node7)
node3.add_child(node8)
node4.add_child(node9) iddfs(root, "F")
iddfs(root, "A")
```

Output:



```
Found
Not found
```

Implement A* search algorithm.

Algorithm:

DATE: _____

Implementing A* search

Shortest path from initial point to goal.

Assume starting point to be '1'
Assume end point to be '4'

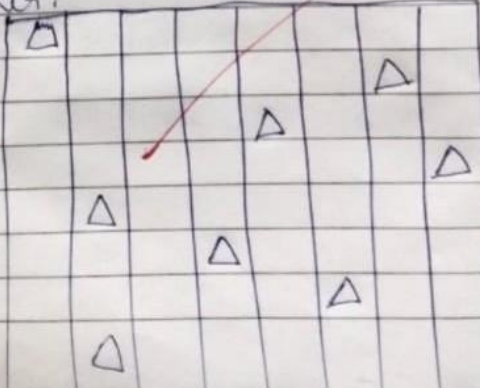
Refer a set of unvisited nodes.
So pick the nodes on basis of priority queue, target which seems the closest.
based on $f(\text{score})$ of the node.

$$f(\text{score}) = g(\text{score}) + h(\text{score})$$

actual Estimated
distance distance

Looking neighbours and keep updating path.
Then track back the nodes travelled to obtain the path.

Goal: Placing all queens on the board such that they do not attack each other.

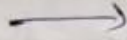


State moves

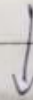
$[(0, 1), (0, -1), (1, 0), (-1, 0)]$

Initial state

1	2	3
8	0	4
7	6	5



1	0	3
8	2	4
7	6	5



8	1	3
0	2	4
7	6	5



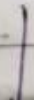
0	1	3
8	2	4
7	6	5



8	1	3
2	0	4
7	6	5



8	1	3
2	4	0
7	6	5



8	0	1
2	4	3
7	6	5



8	1	0
2	4	3
7	6	5



0	8	1
2	4	3
7	6	5



2	8	1
0	4	3
7	6	5

Goal State.

Code:

```
import heapq
import numpy as np

goal = [[2,8,1], [0,4,3], [7,6,5]]
vis = set() q = [] parent_map = {}
move_map = {}

def manhattan(curr):
    ans = 0    pos = {goal[i][j]: (i, j) for i in range(3) for j in range(3)}
    for i in range(3):    for j in range(3):
        x, y = pos[curr[i][j]]
    ans += abs(i - x) + abs(j - y)
    return ans

def moves(curr,g):
    x, y = [(i, j) for i in range(3) for j in range(3) if curr[i][j] == 0][0]
    poss = [[0, -1, 'left'], [-1, 0, 'up'], [1, 0, 'down'], [0, 1, 'right']]
    for dx, dy, direction in poss:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            curr1 = [row[:] for row in curr]
            curr1[x][y], curr1[nx][ny] = curr1[nx][ny], curr1[x][y]
            tuple_curr1 = tuple(map(tuple, curr1))
            if tuple_curr1 not in vis:
                f=g+1+manhattan(curr1)
                heapq.heappush(q, (f, curr1,g+1))
                vis.add(tuple_curr1)
                parent_map[tuple(map(tuple, curr1))] = curr
                move_map[tuple(map(tuple, curr1))] = direction

def a_star(curr,g):
    vis.add(tuple(map(tuple, curr)))
    if curr == goal:
        return True
    moves(curr,g)
    if q:
        curr = heapq.heappop(q)
    if a_star(curr[1],curr[2]):
        return True
    return False

def display_board(board):
    print("+---+---+---+")
    for row in board:
        print("| " + " | ".join(str(x) if x != 0 else ' ' for x in row) + " |")
    print("+---+---+---+")
```

```

c=[[] for i in range(3)] for
i in range(3):
    print(f"Enter elements of row {i+1}")
    c[i]=list(map(int,input().split())) a_star(c,0)

result_path = [] directions
= [] state = goal while
state:
result_path.append(state)
    directions.append(move_map.get(tuple(map(tuple, state)), None))
state = parent_map.get(tuple(map(tuple, state)))

for ind, (state, direction) in enumerate(reversed(list(zip(result_path, directions)))):
print(f'Step {ind}:')    display_board(state)    if ind==0:
    print("Initial state")
if direction:
    print(f' Move empty space {direction}')
print()

print(f'Steps taken: {len(result_path) - 1}')

```

Output:

```
Enter elements of row 1
1 2 3
Enter elements of row 2
8 0 4
Enter elements of row 3
7 6 5
```

Step 0:

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 8 |   | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+
```

Initial state

Step 1:

```
+---+---+---+
| 1 |   | 3 |
+---+---+---+
| 8 | 2 | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+
```

Move empty space up

Implement Hill Climbing search algorithm to solve N-Queens problem.
Algorithm:

Implementing Hill climbing

Always keep on improving the solution until we reach best one or better than older one.

```
def hill_climbing(n=8):  
    board = [random.randint(0, n-1)  
              for i in range(n)]
```

```
    while True:
```

```
        current_attacks = calculate_  
        attacking_pairs(board)  
        if current_attacks == 0:  
            return board
```

```
        neighbours = []  
        for row in range(n):  
            for col in range(n):  
                if col != board[row]:
```

- Count the no. of pairs of queens attacking each other
- for each queen, we try to move it to every other column in its row.
- choose one with least attacking pairs
- If we find configuration = 0 attacks we have a solution.

Code:

```
import random  
def h(s):  
    h = 0  
    n = len(s)  
    for i in range(n):  
        for j in range(i + 1, n):  
            if s[i] == s[j] or abs(s[i] - s[j]) == abs(i - j):
```

```

        h += 1
    return h

def new(s):    best=s
    for i in range(len(s)):
        for j in range(1,9):
            if j!=s[i]:
                n=s[:i]+[j]+s[i+1:]
            if h(n)<h(best):
                best=n
    return best

def hc():
    curr=[random.randint(1,8) for i in range(8)]
    while True:    ch=h(curr)    curr=new(curr)
    if h(curr)==0:    return curr    if
    h(curr)>=ch:    curr=[random.randint(1,8) for
    i in range(8)]

def print_board(solution):
    print("Solution for 8 Queens Hill climbing is: ",solution)
    if solution is None:
        print("No solution found.")
        return

    board = [['.' for _ in range(8)] for _ in range(8)]

    for row in range(len(solution)):
        col = solution[row] - 1
        board[row][col] = 'Q'

    for row in board:
        print(' '.join(row))
    print_board(hc())

```

Output:

Solution for 8 Queens Hill climbing is: [4, 2, 7, 3, 6, 8, 5, 1]

```
. . . Q . . . .  
. Q . . . . . .  
. . . . . . Q .  
. . Q . . . . .  
. . . . . Q . .  
. . . . . . . Q  
. . . . Q . . .  
Q . . . . . . .
```

Implement A star algorithm to solve N-Queens problem.

Algorithm:

Q Solving 8 Puzzle problem using A*
4 Implementing Iterative Deeping Search Algorithms.

Step1: • Initialize the problem
• Define the initial state in a puzzle of (3x3).
• Define goal state in puzzle of 3x3 determining empty tile $\rightarrow 0$

91

initial state			Final state		
1	2	3	2	8	1
8	0	4	0	4	3
7	6	5	7	6	5

Get Lowest Value using priority queue
Calculating each tile from initial to final

	V	H	Dist	Pri. Q
1	0	2	2	2
2	0	1	1	1
3	1	0	1	1
4	1	1	2	1
8	0	1	1	2
0	0	0	0	
7	0	0	0	0
6	0	0	0	0
5	0	0	0	0

Applying this priority queue.

Code:

```
import heapq
```

```
def h(s):
```

```

    h = 0    n = len(s)    for i in range(n):        for j
in range(i + 1, n):        if s[i] == s[j] or abs(s[i]
- s[j]) == abs(i - j):
            h += 1
return h

def a_star():
    initial_state = []
    q = []    g = 8
    heapq.heappush(q, (h(initial_state), initial_state, g))

    while q:
        f, state, g = heapq.heappop(q)

        if len(state) == 8 and h(state) == 0:
            return state

        for i in range(1, 9):
            if i not in state:
                new_state = state + [i]

                heapq.heappush(q, (h(new_state) + g, new_state, g - 1))

    return None

solution = a_star() print("Solution:",
solution)

```

Output:

```
Solution for 8 Queens A* search is: [1, 5, 8, 6, 3, 7, 2, 4]
Q . . . . . . .
. . . . Q . . .
. . . . . . Q
. . . . Q . .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . . .
```

Implement Simulated Annealing:

Algorithm:

Stimulate Annealing	DATE :
Function $\rightarrow x^2 + 5 \sin x$ function Stimulated Annealing (initial state, initial temperature, cooling-rate iteration) current_state = initial_state best_state = current_state best_cost = objective function (current state) temp = initial temperature while temp > 1: for i = 1 to iterations: new_state = neighbors (current state) curr_cost = objective function (new state) new_cost = objective function (new state) if $\Delta p(\text{curr_cost}, \text{new_cost}, \text{temp}) > \text{Randomly}$ current_state = new_state if new_cost < best_cost: best_state = new_state best_cost = new_cost temp = cooling-rate return (best_state, best_cost) function objective function (state):	for n in state cost = $x^2 + 5 \sin x$ return cost function Neighbors (state) new_state = state.copy() index = Random (0, length (state) - 1) new_state [index] = Random (-1, 1) return new_state function $\Delta p(\text{curr_state}, \text{new_cost}, \text{temp})$ if (new_cost < curr_cost): return else: return $\exp(-(\text{new_cost} - \text{curr_cost}) / \text{temp})$

Code:

```
import random
import math

def sim_anneal(ini, in_temp, max_iter, cool):
    # Initialize current state and best state
    curr_s = ini    best_s = curr_s    best_c =
    obj(best_s)
    temp = in_temp # Set the initial temperature

    # While the temperature is above a threshold
    while temp > 1:
        for i in range(max_iter):
            new_s = neig(curr_s)
            curr_c = obj(curr_s)
            new_c = obj(new_s)

            if ap(curr_c, new_c, temp) > random.random():
                curr_s = new_s # Move to the new state        if
                new_c < best_c:            best_s = new_s
            best_c = new_c
            temp *= cool

    return best_s, best_c

def neig(state):
    new_s = state.copy()    ind =
    random.randint(0, len(state) - 1)
    new_s[ind] += random.uniform(-1, 1)
    return new_s

def obj(state):
    c = 1    for i in state:
    c += i**2 + 2*i + 1
    return c

def ap(curr_c, new_c, temp):
    if new_c < curr_c:
    return 1    else:
        return math.exp((curr_c - new_c) / temp)

print(sim_anneal([1, 2, 3, 4, 5], 1000, 1000, 0.99))
Output:
```

```
The best position found is: [0 8 5 2 6 3 7 4]
The number of queens that are not attacking each other is: 8
```

Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.

Algorithm:

Propositional Logic

PAGE NO:
 DATE:

- ① Alice is mother of Bob
- ② Bob is father of Charlie
- ③ A father is parent
- ④ A mother is parent
- ⑤ All the parents have children
- ⑥ If someone is parent, their children are sibling
- ⑦ Alice is married to David

Hypothesis: Charlie is sibling of Bob.

Propositional Logic

$M(A, B)$: Alice is mother of Bob
 $F(B, C)$: Bob is father of Charlie
 $Parent(x)$: x is parent.
 $child(y, x)$: y is child of x .
 $siblings(x, y)$: x & y are siblings.
 $Married(A, D)$: Alice is married to David
 $Parent(x)$ have

Logical Reasoning:

From state ① & ②
 $M(A, B) \wedge F(B, C) \rightarrow$ Alice is parent

From ② & ③
 $F(B, C) \wedge C(y, x) \rightarrow$ Bob is parent

From ① & ② & ⑦
 $M(A, B) \wedge F(B, C) \wedge x \wedge C(x, y)$
 Bob & Alice are not siblings

Code:

```
from sympy.logic.boolalg import Or, And, Not from
sympy.abc import A, B, C, D, E, F
from sympy import simplify_logic

def is_ entailment(kb, query):

    # Negate the query
    negated_query = Not(query)

    # Add negated query to the knowledge base
    kb_with_negated_query = And(*kb, negated_query)

    # Simplify the combined KB to CNF
    simplified_kb = simplify_logic(kb_with_negated_query, form="cnf")

    # If the simplified KB evaluates to False, the query is entailed
    return simplified_kb == False


# Define a larger Knowledge Base kb
= [
    Or(A, B),      #  $A \vee B$ 
    Or(Not(A), C), #  $\neg A \vee C$ 
    Or(Not(B), D), #  $\neg B \vee D$ 
    Or(Not(D), E), #  $\neg D \vee E$ 
    Or(Not(E), F), #  $\neg E \vee F$ 
    F              #  $F$ 
]
# Query to check
query = Or(C, F) #  $C \vee F$ 

# Check entailment result =
is_ entailment(kb, query)
print(f'Is the query '{query}' entailed by the knowledge base? {'Yes' if result else 'No'})
```

Output:

a	b	c	KB	Query
False	False	False	False	False
False	False	True	False	False
False	True	False	False	True
False	True	True	True	True
True	False	False	True	True
True	False	True	False	True
True	True	False	True	True
True	True	True	True	True

Combinations where both KB and Query are True:

a	b	c	KB	Query
False	True	True	True	True
True	False	False	True	True
True	True	False	True	True
True	True	True	True	True

Implement unification in first order logic.

Algorithm:

PAGE NO :
DATE :

Unification in First order logic

Key conditions

- same predicate symbol is the predicate symbols in the expressions must match
- same number of arguments: the expressions must have an equal number of argument.
- variable conflict resolution: variables cannot take multiple conflicting values
- No conflicting function symbols: Different function symbols cannot identify

Examples:

① Expression A: $\text{knows}(f(x, y), g(z))$
Expression B: $\text{knows}(f(\text{Alice}, \text{Bob}), g(z))$

Steps:

- By comparing the predicates, we arrive that both are knows
- Consider $f(x, y) \rightarrow f(\text{Alice}, \text{Bob})$
 $x = \text{Alice}, y = \text{Bob} \quad g(x) \rightarrow g(z)$
 $z = \text{Alice} \text{ (Since } x = \text{Alice)}$
- $x = \text{Alice}$
 $y = \text{Bob}$
 $z = \text{Alice}$
- Unified expression
 $\text{knows}(f(\text{Alice}, \text{Bob}), g(\text{Alice}))$

Code:

```
import re

def occurs_check(var, x):
    if var == x:        return
    True elif isinstance(x,
list):
        return any(occurs_check(var, xi) for xi in x)
    return False

def unify_var(var, x, subst):
    if var in subst:
    return unify(subst[var], x, subst) elif isinstance(x,
(list, tuple)) and tuple(x) in subst:
        return unify(var, subst[tuple(x)], subst)
    elif occurs_check(var, x):        return
    "FAILURE" else:
        subst[var] = tuple(x) if isinstance(x, list) else x
    return subst

def unify(x, y, subst=None):
    if subst is
None:        subst = {}
    if x == y:        return
    subst elif isinstance(x, str) and x.islower():
    return unify_var(x, y, subst) elif
isinstance(y, str) and y.islower():        return
    unify_var(y, x, subst) elif isinstance(x, list)
and isinstance(y, list):
        if len(x) != len(y):
        return "FAILURE"
        if
x[0] != y[0]:
            return "FAILURE"
        for xi, yi in zip(x[1:], y[1:]):
        subst = unify(xi, yi, subst)
        if subst == "FAILURE":
        return "FAILURE"
        return
    subst else:
        return "FAILURE"

def unify_and_check(expr1, expr2):
    result = unify(expr1, expr2)
    if
result == "FAILURE":
        return False, None
    return True, result

def display_result(expr1, expr2, is_unified, subst):
    print("Expression 1:", expr1)
    print("Expression
```

```

2:", expr2)  if not is_unified:      print("Result:
Unification Failed")  else:
    print("Result: Unification Successful")
    print("Substitutions:", {k: list(v) if isinstance(v, tuple) else v for k, v in subst.items()})

def parse_input(input_str):
    input_str = input_str.replace(" ", "")  def
    parse_term(term):      if '(' in term:      match =
    re.match(r'([a-zA-Z0-9_]+)(.*)', term)      if
    match:
        predicate = match.group(1)
    arguments_str = match.group(2)
        arguments = [parse_term(arg.strip()) for arg in arguments_str.split(',')]
    return [predicate] + arguments      return term
    return parse_term(input_str)

def main():  while True:      expr1_input = input("Enter the first
expression (e.g., p(x, f(y))): ")      expr2_input = input("Enter the
second expression (e.g., p(a, f(z))): ")      expr1 =
    parse_input(expr1_input)      expr2 = parse_input(expr2_input)
        is_unified, result = unify_and_check(expr1, expr2)      display_result(expr1, expr2,
    is_unified, result)      another_test = input("Do you want to test another pair of expressions?
    (yes/no):
    ").strip().lower()      if
    another_test != 'yes':
        break

if __name__ == "__main__":
    main()

```

Output:

```
Enter the first expression (e.g., p(x, f(y))): p(b,x,f(g(z)))
Enter the second expression (e.g., p(a, f(z))): p(z,f(y),f(y))
Expression 1: ['p', '(b', 'x', ['f', '(g(z))'])]
Expression 2: ['p', '(z', ['f', '(y)'], ['f', '(y)'])]
Result: Unification Successful
Substitutions: {'(b': '(z', 'x': ['f', '(y)'], '(g(z))': '(y))'}
Do you want to test another pair of expressions? (yes/no): yes
Enter the first expression (e.g., p(x, f(y))): p(x,h(y))
Enter the second expression (e.g., p(a, f(z))): p(a,f(z))
Expression 1: ['p', '(x', ['h', '(y)'])]
Expression 2: ['p', '(a', ['f', '(z)'])]
Result: Unification Failed
Do you want to test another pair of expressions? (yes/no): yes
Enter the first expression (e.g., p(x, f(y))): p(f(a),g(y))
Enter the second expression (e.g., p(a, f(z))): p(x,x)
Expression 1: ['p', '(f(a)', ['g', '(y)'])]
Expression 2: ['p', '(x', 'x)']
Result: Unification Successful
Substitutions: {'(f(a)': '(x', 'x)': ['g', '(y)')]
Do you want to test another pair of expressions? (yes/no): no
```

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.

Algorithm:

First Order Logic: Forward Chaining

PAGE NO: _____
DATE: _____

Step 1: Initialize Facts
Create a dictionary or data structure to store facts:
facts = { "American(Robert)": True, "Hostile(country)": True, "Sells(Robert, Missiles, Country)": True }

Step 2: Define the Rule
Rule: $\text{Criminal}(x) \leftarrow \text{American}(x) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z)$
inputs:
• name of the person (x), Fact dictionary
Logic:
Check if all conditions are satisfied
• American(x) is True
• Sells(x, y, z) is True for some y, z
• Hostile(z) is True

Step 3: Apply the Rule:
For x = Robert, evaluate:
• American(Robert)
• Sells(Robert, Missiles, Country A)
• Hostile(country A)
If all conditions are True, deduces $\text{criminal}(\text{Robert}) = \text{True}$.

Step 4: Output Result
If $\text{criminal}(\text{Robert})$ is True
print: "Robert is criminal"
else
print: "Robert is not criminal"

Code:

```
KB = set()

KB.add('American(Robert)')
KB.add('Enemy(America, A)')
KB.add('Missile(T1)')
KB.add('Owns(A, T1)')

def modulus_ponens(fact1, fact2, conclusion):
    if fact1 in KB and fact2 in KB:
        KB.add(conclusion)    print(f'Inferred:
                               {conclusion}')

def forward_chaining():
    if 'Missile(T1)' in KB:
        KB.add('Weapon(T1)')
        print(f'Inferred: Weapon(T1)')

    if 'Owns(A, T1)' in KB and 'Weapon(T1)' in KB:
        KB.add('Sells(Robert, T1, A)')
        print(f'Inferred: Sells(Robert, T1, A)')

    if 'Enemy(America, A)' in KB:
        KB.add('Hostile(A)')    print(f'Inferred:
        Hostile(A)')    if 'American(Robert)' in
        KB and 'Weapon(T1)' in KB and
        'Sells(Robert, T1, A)' in KB and
        'Hostile(A)' in KB:
            KB.add('Criminal(Robert)')
            print("Inferred: Criminal(Robert)")

    if 'Criminal(Robert)' in KB:
        print("Robert is a criminal!")
    else:
        print("No more inferences can be made.")

forward_chaining()
```

Output:

```
Inferred: Weapon(T1)
Inferred: Sells(Robert, T1, A)
Inferred: Hostile(A)
Inferred: Criminal(Robert)
Robert is a criminal!
```

Create a knowledge base consisting of first order logic statements and prove the given query using Resolution.

Algorithm:

Resolution in First Order Logic

Step 1:
Express Knowledge Base (KB) in first OL

① Premises:

- ①. $\forall x \text{ Food}(x) \rightarrow \text{likes}(\text{John}, x)$
(John likes all kind of food)
- ②. $\text{Food}(\text{Apple}) \wedge \text{Food}(\text{vegetables})$
(Apples and vegetables are food)
- ③. $\forall x \forall y (\text{Eats}(x, y) \wedge \neg \text{killed}(y) \rightarrow \text{Food}(y))$
Anything anyone eats & is not killed is food
- ④. $\text{Eats}(\text{Anil}, \text{Peanuts}) \wedge \neg \text{killed}(\text{Anil})$
(Anil eats peanuts and still alive)
- ⑤. $\forall x \text{ Eats}(\text{Anil}, x) \rightarrow \text{Eats}(\text{Harry}, x)$
Harry eats everything that Anil eats.
- ⑥. $\forall x \text{ Alive}(x) \rightarrow \neg \text{killed}(x)$
Anyone who is alive is not killed.
- ⑦. $\neg \text{killed}(x) \rightarrow \text{Alive}(x)$
Anyone who is not killed is Alive.

⑧ Goal: $\text{likes}(\text{John}, \text{Peanuts})$

⑨ Apply resolution

- ① From ④:
 $\text{Eats}(\text{Anil}, \text{Peanuts}) \wedge \neg \text{killed}(\text{Anil})$
- ② From ③:
Substitute $y = \text{Anil}, x = \text{Peanuts}$:
 $\neg \text{Eats}(\text{Anil}, \text{Peanuts}) \vee \neg \text{killed}(\text{Anil})$
 $\vee \text{Food}(\text{Peanuts})$
- ③ From ①:
Substitute $x = \text{Peanuts}$.

Code:

```
# Define the knowledge base (KB)
KB = {
    # Rules and facts
    "philosopher(X)": "human(X)", # Rule 1: All philosophers are humans
    "human(Socrates)": True, # Socrates is human (deduced from philosopher)
    "teachesAtUniversity(X)": "philosopher(X) or scientist(X)", # Rule 2
    "some(philosopher, not scientist)": True, # Rule 3: Some philosophers are not scientists
    "writesBooks(X)": "teachesAtUniversity(X) and philosopher(X)", # Rule 4
    "philosopher(Socrates)": True, # Fact: Socrates is a philosopher
    "teachesAtUniversity(Socrates)": True, # Fact: Socrates teaches at university }

# Function to evaluate a predicate based on the KB def
resolve(predicate): # If it's a direct fact in KB if
    predicate in KB and isinstance(KB[predicate], bool):
        return KB[predicate]

    # If it's a derived rule
    if predicate in KB:
        rule = KB[predicate]

        if " and " in rule: # Handle conjunction
            sub_preds = rule.split(" and ")
            return all(resolve(sub.strip()) for sub in sub_preds)
        elif " or " in rule: # Handle disjunction
            sub_preds = rule.split(" or ")
            return any(resolve(sub.strip()) for sub in sub_preds)
        elif "not " in rule: # Handle negation
            sub_pred = rule[4:] # Remove "not "
            return not resolve(sub_pred.strip())
        else: # Handle single predicate
            return resolve(rule.strip())

    # If the predicate contains variables
    if "(" in predicate:
        func, args = predicate.split("(")
        args = args.strip(")").split(", ") #
        # Handle philosopher and human link
        if func == "philosopher":
            return resolve(f'human({args[0]})') #
        # Handle writesBooks rule explicitly
        if func == "writesBooks":
            return resolve(f'teachesAtUniversity({args[0]})') and resolve(f'philosopher({args[0]})')
    # Default to False if no rule or fact applies
    return False
```

```
# Query to check if Socrates writes books query
= "writesBooks(Socrates)"
result = resolve(query)

# Print the result print("Output:
1BM22CS200")
print(f'Does Socrates write books? {'Yes' if result else 'No'}')
```

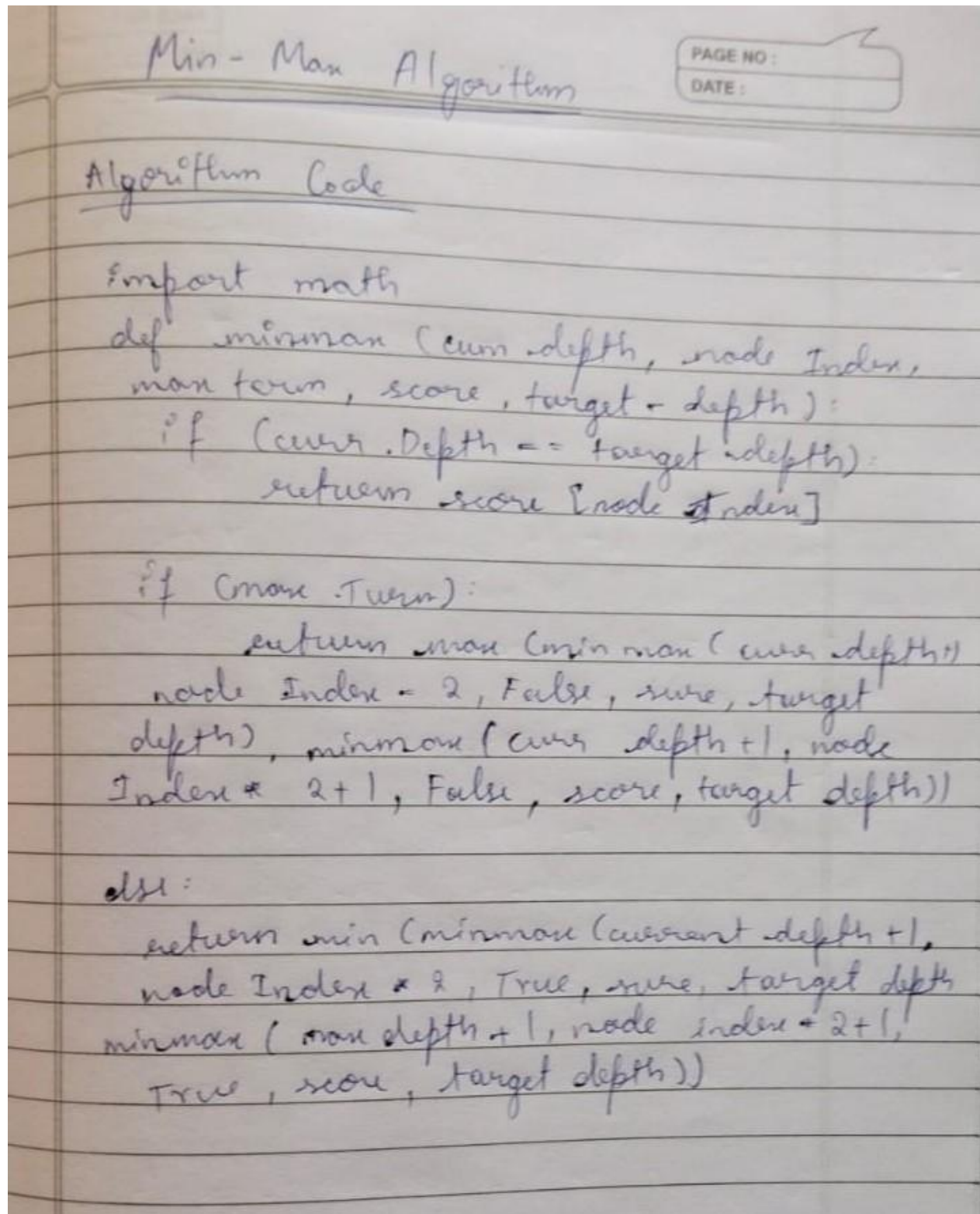
Output:

Does Socrates write books? YES

```
Does John like peanuts? Yes
```

Implement MinMax Algorithm for TicTacToe.

Algorithm:



Code:

```
import math
```

```
def printBoard(board):    for row in board:        print(" |"  
".join(cell if cell != "" else " " for cell in row))        print("-"  
" * 9)
```

```
def evaluateBoard(board):    for row in board:        if  
row[0] == row[1] == row[2] and row[0] != "":  
            return 10 if row[0] == 'X' else -10        for col in range(3):            if  
board[0][col] == board[1][col] == board[2][col] and board[0][col] != "":  
                return 10 if board[0][col] == 'X' else -10            if board[0][0] ==  
board[1][1] == board[2][2] and board[0][0] != "":  
                return 10 if board[0][0] == 'X' else -10            if board[0][2] ==  
board[1][1] == board[2][0] and board[0][2] != "":  
                return 10 if board[0][2] == 'X' else -10  
    return 0
```

```
def isDraw(board):  
    for row in board:  
        if "" in row:  
            return False        return  
    True
```

```
def minimax(board, depth, isMaximizing):  
    score = evaluateBoard(board)  
    if score == 10 or score == -10:  
        return score  
    if isDraw(board):
```

```

    return 0

    if isMaximizing:
        bestScore = -math.inf
    for i in range(3):
        for j
        in range(3):
            if
            board[i][j] == "":
                board[i][j] = 'X'

                score = minimax(board, depth + 1, False)

            board[i][j] = ""
            bestScore =
            max(bestScore, score)
            return bestScore
        else:
            bestScore = math.inf
    for i in range(3):
        for j
        in range(3):
            if
            board[i][j] == "":
                board[i][j] = 'O'

                score = minimax(board, depth + 1, True)

            board[i][j] = ""
            bestScore =
            min(bestScore, score)
            return bestScore

def findBestMove(board):
    bestValue = -math.inf
    bestMove = (-1, -1)
    for
    i in range(3):
        for j in
        range(3):
            if
            board[i][j] == "":
                board[i][j] = 'X'

```

```

        moveValue = minimax(board, 0, False)
board[i][j] = ""          if moveValue >
bestValue:                bestMove = (i, j)
bestValue = moveValue     return bestMove

def playGame():
    board = [["" for _ in range(3)] for _ in range(3)]
    print("Tic Tac Toe!")    print("You are 'O'. The AI
is 'X'.")    printBoard(board)

    while True:
while True:
try:
        row, col = map(int, input("Enter your move (row and column: 0, 1, or 2): ").split())
if board[row][col] == "":        board[row][col] = 'O'
        break
else:
        print("Cell is already taken. Choose another.")
except (ValueError, IndexError):
        print("Invalid input. Enter row and column as two numbers between 0 and 2.")

    print("Your move:")
    printBoard(board)

    if evaluateBoard(board) == -10:
        print("You win!")
    break    if
isDraw(board):

```

```
        print("It's a draw!")
    break

    print("AI is making its move...")
    bestMove = findBestMove(board)
    board[bestMove[0]][bestMove[1]] = 'X'

    print("AI's move:")
    printBoard(board)

    if evaluateBoard(board) == 10:
        print("AI wins!")
    break
    if
isDraw(board):
    print("It's a draw!")
    break

playGame()
```

Output:

```

Tic Tac Toe!
You are 'O'. The AI is 'X'.
|  |
-----
|  |
-----
|  |
-----
Enter your move (row and column: 0, 1, or 2): 2 2
Your move:
|  |
-----
|  |
-----
|  | O
-----
AI is making its move...
AI's move:
|  |
-----
| X |
-----
|  | O
-----
Enter your move (row and column: 0, 1, or 2): 0 0
Your move:
O |  |
-----
| X |
-----
|  | O
-----
AI is making its move...
AI's move:
O | X |
-----
| X |
-----
|  | O
-----
Enter your move (row and column: 0, 1, or 2): 2 1
Your move:
O | X |
-----
| X |
-----
| O | O
-----
AI is making its move...

```

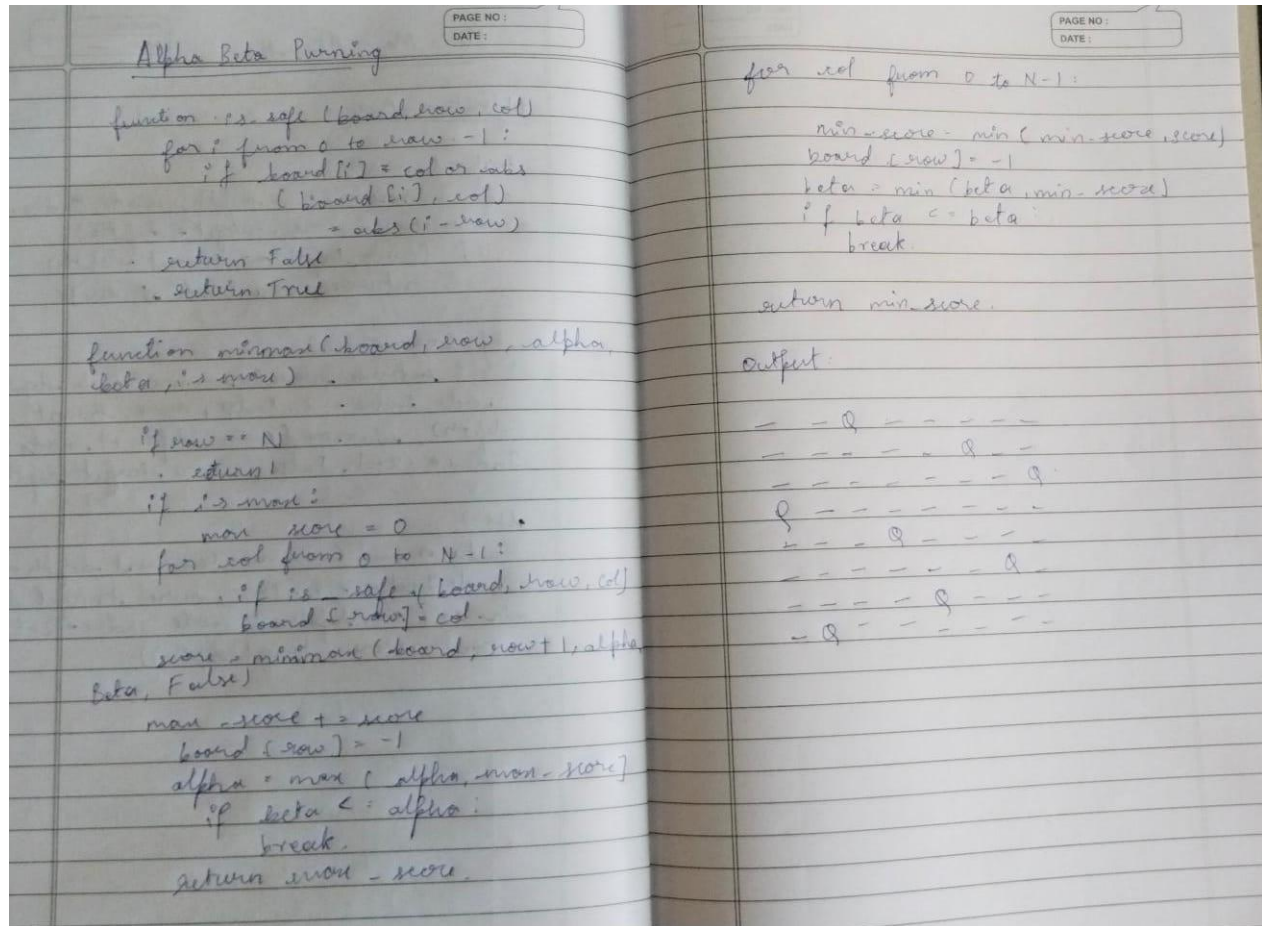
```

AI is making its move...
AI's move:
O | X |
-----
| X |
-----
X | O | O
-----
Enter your move (row and column: 0, 1, or 2): 0 2
Your move:
O | X | O
-----
| X |
-----
X | O | O
-----
AI is making its move...
AI's move:
O | X | O
-----
| X | X
-----
X | O | O
-----
Enter your move (row and column: 0, 1, or 2): 1 0
Your move:
O | X | O
-----
O | X | X
-----
X | O | O
-----
It's a draw!

```

Implement Alpha-Beta Pruning for 8Queens.

Algorithm:



Code:

```

def is_valid(board, row, col):

    for i in range(row):
        if board[i] == col or \
abs(board[i] - col) == abs(i - row):
            return False
    return True

def alpha_beta(board, row, alpha, beta, isMaximizing):

    if row == len(board):
        return 1

    if isMaximizing:
        max_score = 0
        for col in range(len(board)):
            if
    
```

```

is_valid(board, row, col):
    board[row] = col
    max_score += alpha_beta(board, row + 1, alpha, beta, False)
    board[row] = -1
    alpha = max(alpha, max_score)
    if beta <= alpha:
        break
    return max_score
else:
    min_score = float('inf')
    for col in range(len(board)):
        if is_valid(board, row, col):
            board[row] = col
            min_score = min(min_score, alpha_beta(board, row + 1, alpha, beta, True))
    board[row] = -1
    beta = min(beta, min_score)
    if beta <= alpha:
        break
    return min_score

def solve_8_queens():
    board = [-1] * 8
    alpha = -float('inf')
    beta = float('inf')
    return alpha_beta(board, 0, alpha, beta, True)

solutions = solve_8_queens()
print(f'Number of solutions for the 8 Queens problem: {solutions}')

```

Output:

Number of solutions for the 8 Queens problem: 92

Solution 1:

```
Q . . . . . . .  
. . . . Q . . .  
. . . . . . Q  
. . . . . Q . .  
. . Q . . . . .  
. . . . . . Q .  
. Q . . . . . .  
. . . Q . . . .
```

Solution 2:

```
Q . . . . . . .  
. . . . . Q . .  
. . . . . . Q  
. . Q . . . . .  
. . . . . . Q .  
. . . Q . . . .  
. Q . . . . . .  
. . . . Q . . .
```

Solution 3:

```
Q . . . . . . .  
. . . . . . Q .  
. . . . Q . . .  
. . . . . Q . .  
. . . . . . Q  
. Q . . . . . .  
. . . . Q . . .  
. . Q . . . . .
```

Solution 4:

```
Q . . . . . . .  
. . . . . Q . .  
. . . . Q . . .  
. . . . . . Q  
. Q . . . . . .  
. . . Q . . . .  
. . . . . Q . .  
. . Q . . . . .
```